

N-Queens Problem — Complete DSA Notes

1. Problem Statement

Place N queens on an:

`N × N chessboard`

such that:

- no two queens attack each other

A queen can attack:

- horizontally
- vertically
- diagonally

Goal:

`Find all valid arrangements`

or sometimes:

- count total solutions
- print one solution

2. Example (4 Queens)

Board:

`. Q . .`

```
. . . Q
Q . . .
. . Q .
```

This is valid because:

- no queens share same row
- no queens share same column
- no queens share same diagonal

3. Why This Problem is Important

N-Queens teaches:

- Backtracking
- Recursion
- DFS (Depth First Search)
- Constraint checking
- State space tree
- Decision making
- Optimization/pruning

This is one of the most important recursion problems in DSA.

4. Main Idea

We place queens:

```
row by row
```

Meaning:

```
Row 0 → place queen  
Row 1 → place queen  
Row 2 → place queen  
...
```

At each row:

- try every column
- check if safe
- place queen
- recurse
- remove queen (backtrack)

5. Why Row-by-Row?

Because:

- one queen must exist in each row
- recursion naturally handles rows one by one

So we avoid checking:

- same row conflicts

This is a major optimization.

6. Meaning of Function

```
Nqueen(board,row)
```

means:

```
Try placing queens starting from this row.
```

7. Recursion Structure

Core structure:

```
for(each column){  
    if(safe){  
        place queen  
        recurse to next row  
        remove queen  
    }  
}
```

8. Core Backtracking Formula

Every backtracking problem follows:

```
MAKE CHOICE  
  ↓  
EXPLORE  
  ↓  
UNDO CHOICE
```

In N-Queens:

```
board[row][j]='Q';      // MAKE CHOICE  
  
Nqueen(board,row+1);    // EXPLORE  
  
board[row][j]='.';      // UNDO
```

9. What is Backtracking?

Backtracking means:

```
Try one possibility.  
If it fails later,  
come back and try another.
```

It is basically:

```
Systematic trial and error
```

with recursion.

10. Why Undo is Necessary?

Without:

```
board[row][j]='.';
```

old queens remain on board permanently.

Then future recursive calls see incorrect board state.

Undo restores board to previous state.

11. Why Only Upward Checking?

Because recursion places queens:

```
top → bottom
```

Future rows are still empty.

So checking downward is useless.

We only check previous rows.

12. What isSafe() Checks

Three directions:

A. Vertical Up

```
for(int i=row-1;i>=0;i--)
```

Checks:

```
↑  
↑  
↑
```

No queen should exist in same column.

B. Upper Left Diagonal

```
i-- , j--
```

Movement:

```
↖ ↖ ↖
```

C. Upper Right Diagonal

```
i-- , j++
```

Movement:



13. Why No Horizontal Check?

Because:

- recursion already guarantees
- one queen per row

So impossible to have:

- two queens in same row
-

14. Base Case

```
if(row==board.length)
```

Meaning:

```
All queens successfully placed
```

Then:

- print solution
- count solution
- return true

depending on problem.

15. Recursion Tree Meaning

Each node represents:

placing one queen

Each branch represents:

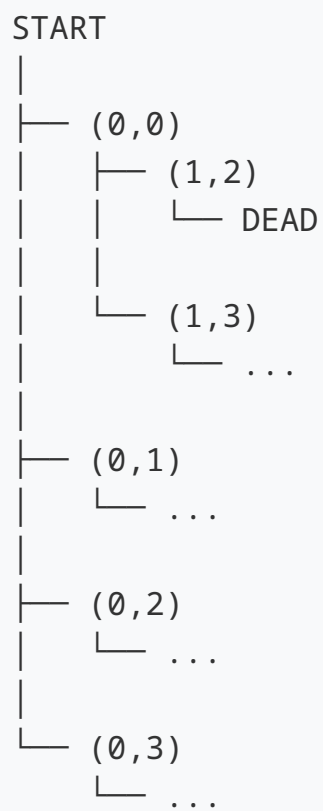
trying another column

Dead branch means:

No safe position available

Then backtracking occurs.

16. Example Recursion Tree (n=4)



Each:

(row, col)

means: queen placed there.

17. Execution Flow

Important:

Recursion does NOT restart loop.

Actual execution:

```
LOOP ITERATION
  ↓
RECURSIVE CALL
  ↓
FULL SUBTREE EXECUTION
  ↓
RETURN
  ↓
UNDO MOVE
  ↓
NEXT LOOP ITERATION
```

18. DFS (Depth First Search)

N-Queens uses:

DFS

Meaning:

Go as deep as possible first.
Then return.

Example:

```
row0
  ↓
row1
  ↓
row2
  ↓
row3
```

Then backtrack upward.

19. Why N=2 and N=3 Have No Solution?

Because:

- queens always attack each other
- impossible arrangement exists

20. Number of Solutions

N	Solutions
1	1
2	0
3	0
4	2
5	10

6	4
7	40
8	92

21. Time Complexity

Worst case:

$O(N!)$

Reason:

Row 1:

- N choices

Row 2:

- roughly N-1 choices

Row 3:

- roughly N-2 choices

So:

$N \times (N-1) \times (N-2) \dots$

$\approx N!$

22. Space Complexity

$O(N)$

due to recursion stack.

23. Important Optimization

Instead of checking board repeatedly:

Use:

- column array
- left diagonal array
- right diagonal array

This reduces:

- safety checking time

Used in optimized solutions.

24. State Space Tree

N-Queens explores:

all possible configurations

This forms a:

State Space Tree

Backtracking prunes invalid branches early.

25. What is Pruning?

Pruning means:

Stop exploring invalid path early

Example:

If queen attack found:

No need to explore deeper

Huge optimization.

26. Mental Model

Think:

"I am standing on one row.
Let me try every possible column.
If future fails,
I will come back and try another."

27. Most Important Interview Answer

Question:

Why use backtracking here?

Answer:

Because one locally valid placement
may create future conflicts.

So we try possibilities recursively
and undo failed choices.

28. Common Mistakes

Mistake 1

Forgetting backtracking:

```
board[row][j]='.';
```

Mistake 2

Checking entire board in isSafe()

Unnecessary.

Only upward directions needed.

Mistake 3

Confusing recursion with loop restart

Loops resume from paused position.

Mistake 4

Using int[][] with 'Q'

Wrong datatype.

Use:

```
char[][]
```

29. Real-Life Analogy

Imagine:

- placing security cameras
- no camera should overlap visibility

You:

- place one
- check conflicts
- if future placement impossible
- remove and reposition

That is backtracking.

30. Core Pattern Recognition

N-Queens belongs to:

Decision Tree Problems

At every level:

- choose one option
 - recursively explore consequences
-

31. Similar Problems

Same pattern used in:

- Sudoku Solver
- Rat in Maze
- Permutations
- Combination Sum
- Word Search

- Subsets
 - Knight Tour
-

32. One-Line Summary

N-Queens uses recursion + backtracking to place queens row by row, e.